OrarioClasse — Secondo passo: il motore degli scambi

1. Obiettivo

Nel primo passo abbiamo costruito il modello dati con le classi `Tessera`, `Posizione` e `Orario`. Ora realizziamo il **motore degli scambi**, senza interfaccia grafica.

Il programma deve ricevere due posizioni, scambiarne il contenuto e **confermare la modifica soltanto se non si creano buchi** nelle lezioni di una giornata.

L'orario comprende 5 giorni, 8 ore per giorno, 34 tessere (ciascuna con materia e aula) e 6 celle vuote. Una cella vuota è rappresentata da `null`. Le lezioni di ciascun giorno devono iniziare alla prima ora e proseguire senza interruzioni; le celle vuote sono ammesse soltanto alla fine della giornata.

2. Organizzazione dei file

```
OrarioClasse/
├── Model/
│   ├── Tessera.cs
│   ├── Posizione.cs
│   └── Orario.cs
├── Engine/
│   ├── ValidatoreOrario.cs
│   └── MotoreScambi.cs
└── Program.cs
```

Le classi del modello conservano i dati; `ValidatoreOrario` verifica la regola dei buchi; `MotoreScambi` gestisce la modifica. Non viene utilizzata alcuna classe WPF.

3. Modello dati del primo passo

Manteniamo la sintassi tradizionale: classi, costruttori, proprietà, metodi, matrice bidimensionale. Per questa versione didattica impostiamo nel file `.csproj`:

```
<Nullable>disable</Nullable>
```

Questo consente di usare `null` con i tipi riferimento senza introdurre ancora la sintassi `Tessera?`.

Tessera.cs

```
namespace OrarioClasse.Model
{
    public class Tessera
    {
        public string Materia { get; set; }
```

```
        public string Aula { get; set; }

        public Tessera(string materia, string aula)
        {
            Materia = materia;
            Aula = aula;
        }
    }
}
```

Posizione.cs

```
namespace OrarioClasse.Model
{
    public class Posizione
    {
        public int Giorno { get; set; }
        public int Ora { get; set; }

        public Posizione(int giorno, int ora)
        {
            Giorno = giorno;
            Ora = ora;
        }
    }
}
```

Gli indici partono da zero: `Giorno` va da 0 a 4, `Ora` da 0 a 7.

Orario.cs

```
namespace OrarioClasse.Model
{
    public class Orario
    {
        private Tessera[,] celle;

        public Orario()
        {
            celle = new Tessera[5, 8];
        }

        public Tessera Leggi(Posizione posizione)
        {
            return celle[posizione.Giorno, posizione.Ora];
        }

        public void Scrivi(Posizione posizione, Tessera tessera)
        {
```

```
            celle[posizione.Giorno, posizione.Ora] = tessera;
        }
    }
}
```

Il metodo `Scrivi` rimane pubblico per semplicità didattica. Nell'applicazione finale la GUI non dovrà usarlo per effettuare scambi: dovrà rivolgersi al motore.

4. La regola: nessun buco

Esempio di giornata **valida** (`X` = lezione, `-` = cella vuota):

```
Ora:  1 2 3 4 5 6 7 8
      X X X X X X - -
```

Esempio di giornata **non valida**:

```
Ora:  1 2 3 4 5 6 7 8
      X X X - X X - -
```

La quarta ora è vuota ma la quinta e la sesta sono occupate.

L'algoritmo di validazione esamina ogni giornata da sinistra a destra. Quando incontra la prima cella vuota, tutte le celle successive devono essere vuote.

5. ValidatoreOrario.cs

```
using OrarioClasse.Model;

namespace OrarioClasse.Engine
{
    public class ValidatoreOrario
    {
        public bool SenzaBuchi(Orario orario)
        {
            for (int giorno = 0; giorno < 5; giorno++)
            {
                bool trovataCellaVuota = false;

                for (int ora = 0; ora < 8; ora++)
                {
                    Posizione posizione = new Posizione(giorno, ora);
                    Tessera tessera = orario.Leggi(posizione);

                    if (tessera == null)
                    {
                        trovataCellaVuota = true;
```

```
                    }
                    else if (trovataCellaVuota)
                    {
                        return false;
                    }
                }
            }

            return true;
        }
    }
}
```

Spiegazione

- Il ciclo esterno visita i cinque giorni.
- La variabile `trovataCellaVuota` viene inizializzata a `false` all'inizio di ogni giorno.
- Il ciclo interno visita le otto ore.
- Se una cella è vuota, la variabile diventa `true`.
- Se dopo una cella vuota compare una lezione, il metodo restituisce immediatamente `false`.
- Se il controllo termina senza trovare buchi, restituisce `true`.

Il validatore **non modifica** l'orario: legge soltanto i dati e restituisce un risultato logico.

6. MotoreScambi.cs

```
using OrarioClasse.Model;

namespace OrarioClasse.Engine
{
    public class MotoreScambi
    {
        private ValidatoreOrario validatore;

        public MotoreScambi()
        {
            validatore = new ValidatoreOrario();
        }

        public bool Scambia(Orario orario,
                            Posizione prima,
                            Posizione seconda)
        {
            // Conserva i contenuti originali.
            Tessera tessera1 = orario.Leggi(prima);
            Tessera tessera2 = orario.Leggi(seconda);

            // Esegue provvisoriamente lo scambio.
            orario.Scrivi(prima, tessera2);
            orario.Scrivi(seconda, tessera1);
```

```
            // Se il nuovo orario è valido, conferma lo scambio.
            if (validatore.SenzaBuchi(orario))
            {
                return true;
            }

            // Altrimenti ripristina la situazione precedente.
            orario.Scrivi(prima, tessera1);
            orario.Scrivi(seconda, tessera2);
            return false;
        }
    }
}
```

Spiegazione

Il metodo `Scambia` riceve l'orario e due posizioni. Salva in due variabili temporanee i riferimenti alle tessere (che possono essere `null`), esegue lo scambio e chiama il validatore.

Se il controllo restituisce `true`, lo scambio resta applicato. Se restituisce `false`, il motore rimette le tessere nelle posizioni originali.

Importante: uno scambio fra due celle occupate non crea buchi in un orario inizialmente valido. Lo scambio con una cella vuota, invece, potrebbe crearli e deve essere controllato. Scambiare due celle vuote non modifica l'orario: questa versione lo consente.

7. Programma console di prova

Il seguente esempio prepara un orario valido con **7 lezioni dal lunedì al giovedì e 6 lezioni il venerdì** (7 + 7 + 7 + 7 + 6 = 34).

```
using System;
using OrarioClasse.Model;
using OrarioClasse.Engine;

class Program
{
    static void Main()
    {
        Orario orario = new Orario();
        int numeroTessera = 1;

        for (int giorno = 0; giorno < 5; giorno++)
        {
            int lezioni = 7;
            if (giorno == 4)
            {
                lezioni = 6;
            }
```

```
            for (int ora = 0; ora < lezioni; ora++)
            {
                Tessera tessera = new Tessera(
                    "Materia " + numeroTessera,
                    "Aula " + numeroTessera);

                orario.Scrivi(new Posizione(giorno, ora), tessera);
                numeroTessera++;
            }
        }

        ValidatoreOrario validatore = new ValidatoreOrario();
        MotoreScambi motore = new MotoreScambi();

        Console.WriteLine("Orario iniziale valido: " +
                          validatore.SenzaBuchi(orario));

        // Caso valido: ultima lezione di lunedì
        // nella prima cella vuota di martedì.
        bool esito1 = motore.Scambia(
            orario,
            new Posizione(0, 6),
            new Posizione(1, 7));
        Console.WriteLine("Primo scambio: " + esito1);

        // Caso non valido: si tenta di spostare
        // la prima lezione di mercoledì nell'ottava ora.
        bool esito2 = motore.Scambia(
            orario,
            new Posizione(2, 0),
            new Posizione(2, 7));
        Console.WriteLine("Secondo scambio: " + esito2);

        Console.WriteLine("Orario finale valido: " +
                          validatore.SenzaBuchi(orario));
    }
}
```

Output previsto

```
Orario iniziale valido: True
Primo scambio: True
Secondo scambio: False
Orario finale valido: True
```

Nel primo scambio il lunedì passa da sette a sei lezioni e il martedì da sette a otto: nessun buco. Nel secondo si libererebbe la prima ora di mercoledì lasciando occupate quelle successive: il motore rifiuta e ripristina.

8. Casi da verificare

Situazione	Risultato previsto
Due celle occupate di un orario valido	Scambio autorizzato
Ultima lezione di un giorno ↔ prima cella vuota di un altro giorno	Autorizzato, se entrambe le giornate restano senza buchi
Lezione intermedia ↔ cella vuota	Rifiutato se si crea un buco
Due celle vuote	Autorizzato, senza cambiamenti
Stessa posizione scelta due volte	Nessun cambiamento; questa versione restituisce `true` se l'orario è valido

9. Concetti di programmazione utilizzati

- **Composizione:** `MotoreScambi` contiene un oggetto `ValidatoreOrario`.
- **Incapsulamento:** la matrice è privata nella classe `Orario`.
- **Parametri:** i metodi ricevono oggetti `Orario` e `Posizione`.
- **Valore di ritorno `bool`:** `true` significa controllo superato, `false` significa controllo fallito.
- **Cicli annidati:** il validatore attraversa giorni e ore.
- **Variabili temporanee:** consentono di ripristinare le tessere originali.
- **`null`:** rappresenta una cella priva di lezione.
- **Separazione delle responsabilità:** il validatore controlla, il motore modifica, il modello conserva.

10. Limiti consapevoli e sviluppi successivi

Questa è una versione didattica minima. Si assume che l'orario iniziale sia valido e che le posizioni ricevute siano comprese nei limiti della matrice. Non controlliamo ancora la correttezza delle stringhe `Materia` e `Aula`, il numero esatto di tessere durante il caricamento, né vincoli aggiuntivi come disponibilità di aule e docenti. La GUI non è ancora presente.

Il metodo `Scrivi` è pubblico per semplicità: **non è ancora impedito ad altre parti del programma di modificare l'orario senza autorizzazione**. In un successivo miglioramento dell'architettura limiteremo questa possibilità.

Passo successivo

Realizzare l'interfaccia **WPF/XAML** con una `Grid` di 40 pulsanti, generati dinamicamente. La GUI dovrà selezionare due posizioni, chiamare `MotoreScambi.Scambia` e aggiornare la visualizzazione solo in base all'esito.